

PCIe Flit CRC Engine

1. Overview	2
2. Shared Package and Parameters (crc_pkg.sv)	2
3. Configuration Files	2
3.1. crc_config_512.svh	2
3.2. crc_config_1024.svh	2
3.3. crc_config.svh	3
4. CRC Core Math (crc_calc.sv)	3
4.1. gf256_mult (Galois Field Polynomial Multiplication)	3
4.2. crc_byte_step (Single-Byte CRC Update)	3
4.3. crc_calc (Multi-Byte Block Processing Loop)	3
5. 512-bit Configuration	4
5.1. Implementation Logic	4
5.2. Waveforms	4
6. 1024-bit Configuration	5
6.1. 1024-bit Configuration without split-processing	5
6.1.1. Implementation Logic	5
6.1.2. Waveforms	5
6.2. 1024-bit Configuration with split-processing	6
6.2.1. Implementation Logic	6
6.2.2. Waveforms	7
7. Golden Model (crc_golden_model.sv)	7
8. Summary of differences between 512-bit vs. 1024-bit (Split vs No Split) configurations	8

1. Overview

This document describes the architecture of a synthesizable CRC calculator that computes a 64-bit CRC for a 2048-bit flit. The design supports the following three operating modes:

1. **512-bit input per cycle:** The 2048-bit flit is processed over 4 cycles.
2. **1024-bit input per cycle:** The 2048-bit flit is processed over 2 cycles.
3. **1024-bit input per cycle with two-stage processing:** Each 1024-bit input is divided into two processing stages to reduce the combinational logic depth.

The design is required to support at least one of the first two modes. The third mode is optionally supported by a macro.

2. Shared Package and Parameters (crc_pkg.sv)

Control signals and data are grouped into two packed structs rather than passing loose signals:

1. **i_flit_s (input side)** → start_of_packet, i_data_valid, data_in[DATA_W:0], end_of_packet (DATA_W is set by crc_config.svh)
2. **o_flit_s (output side)** → o_data_valid, data_out[2047:0]

Key parameters:

- FLIT_WIDTH = 2048
- TOTAL_DATA_WIDTH = 1936 (242 bytes of protected payload as per PCIe Appendix K)
- RESERVED_BITS_WIDTH = FLIT_WIDTH – TOTAL_DATA_WIDTH
- CRC_BITS_WIDTH = 64 (as per PCIe Appendix K)

3. Configuration Files

3.1. crc_config_512.svh

This file contains two macros that define the 512-bit datapath:

1. ``INPUT_DATA_BUS_WIDTH = 512`
2. ``NO_OF_CYCLES_PER_FLIT = 4`

3.2. crc_config_1024.svh

This file contains a macro that defines whether the optional split-processing for the 1024-bit configuration is supported, ``SPLIT_1024_CONFIG/`NO_SPLIT_1024_CONFIG`.

If ``SPLIT_1024_CONFIG` was defined, the following macros are defined:

1. ``INPUT_DATA_BUS_WIDTH = 1024`
2. ``NO_OF_CYCLES_PER_FLIT = 2`
3. ``SPLIT_DATA_WIDTH = 512`
4. ``NO_OF_CYCLES_PER_SPLIT_DATA = 4`

If ``NO_SPLIT_1024_CONFIG` was defined, only the first two macros are defined.

3.3. `crc_config.svh`

This file includes either `crc_config_1024.svh` or `crc_config_512.svh` according to a macros that defines the configuration, `CONFIG_1024/CONFIG_512`.

4. CRC Core Math (`crc_calc.sv`)

The 64-bit CRC uses 3 nested functions, where each function calls the one below it:

4.1. `gf256_mult` (Galois Field Polynomial Multiplication)

This function multiplies two 8-bit numbers over Galois Field using CRC finite-field math ($GF(2^8)$). It serves as the core math engine used to apply generator constants (G0-G7) to incoming data. Its calculation is based on the PCIe specification document section 4.2.3.4.4 (generator constants G0-G7, primitive polynomial $x^8 + x^5 + x^3 + x + 1$).

4.2. `crc_byte_step` (Single-Byte CRC Update)

This function updates the overall 64-bit CRC state with one new incoming data byte. It performs a single sequential iteration of the PCIe CRC algorithm, where it:

1. XORs the top byte of the current CRC state with the incoming data byte to create a feedback byte.
2. Multiplies the feedback byte by generator coefficients (G0-G7) using `gf256_mult`
3. Shifts the existing state bytes down and XORs them with these new products.

4.3. `crc_calc` (Multi-Byte Block Processing Loop)

This function takes a multi-byte block of incoming packet data and computes the accumulated CRC over those valid bytes. It starts from an initial seed (`state_in`), it loops byte-by-byte through the incoming data word and repeatedly calls `crc_byte_step` for every valid byte.

5. 512-bit Configuration

5.1. Implementation Logic

In CONFIG_512, one 512-bit chunk of the flit arrives per cycle. The full 2048-bit flit therefore takes 4 cycles to arrive. Each cycle:

- `crc_state` is updated in one call to `crc_calc(crc_state, data_in, 512/8)`, except for the last cycle where the `num_valid_bytes` is $(512 - \text{RESERVED_BITS_WIDTH})/8$ to avoid calculating the CRC for the reserved bits at the end of each flit.
- On each data cycle, `data_out`'s payload field is written, 512 bits at a time, directly from `data_in` (minus reserved bits).
- On one extra “assembly” cycle after the 4 data cycles, `crc_state` is inserted into the assigned bits for CRC in `data_out` and `o_data_valid` is asserted.

This architecture only accepts one flit's worth of data every 5 cycles total (4 data + 1 assembly).

5.2. Waveforms

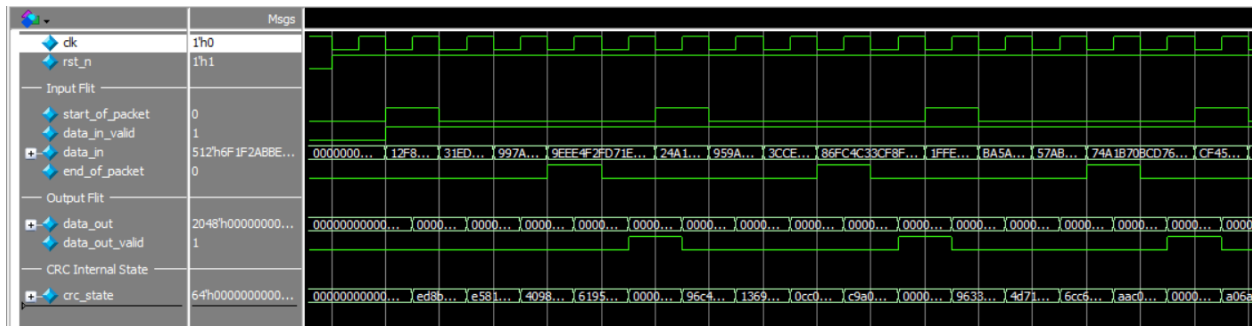


Figure 1: Three consecutive flits in 512-bit configuration

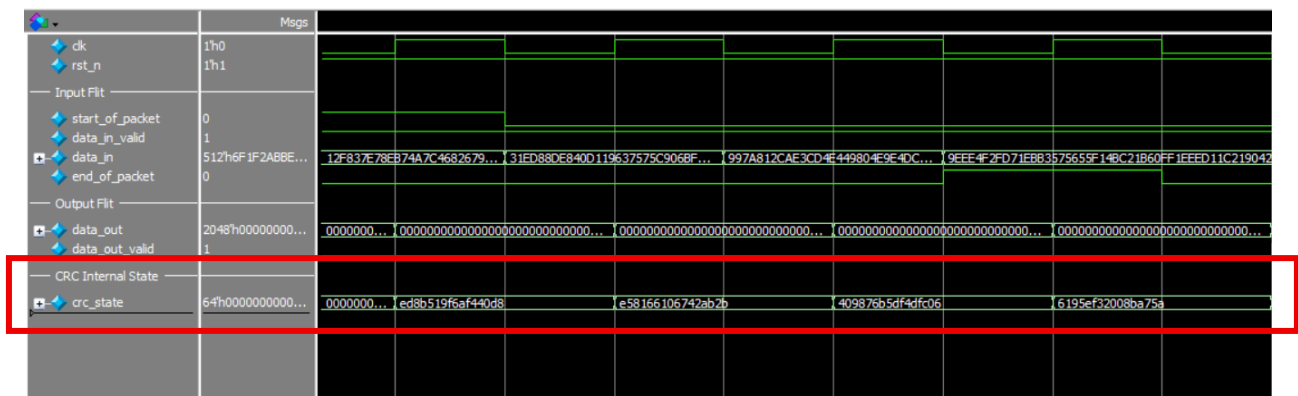
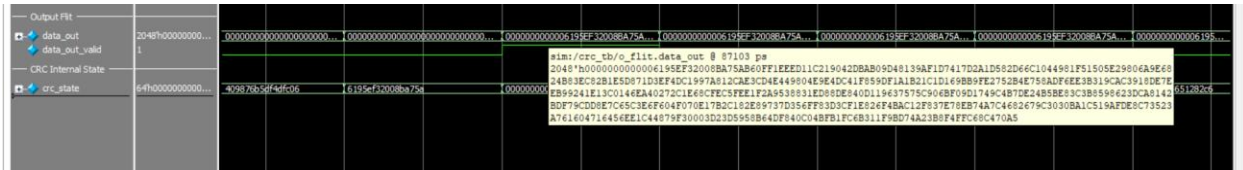


Figure 2: Internal CRC states for one flit in 512-bit configuration



6. 1024-bit Configuration

The 1024-bit configuration, two configurations are implemented, controlled by config macros `SPLIT 1024 CONFIG/`NO SPLIT 1024 CONFIG.

6.1. 1024-bit Configuration without split-processing

6.1.1. Implementation Logic

The 1024-bit configuration is similar to the 512-bit configuration, except for the value of the macros controlling the process. In CONFIG_1024, one 1024-bit chunk of the flit arrives per cycle. The full 2048-bit flit therefore takes 2 cycles to arrive. Each cycle:

- `crc_state` is updated in one call to `crc_calc(crc_state, data_in, 1024/8)`, except for the last cycle where the `num_valid_bytes` is $(1024 - \text{RESERVED_BITS_WIDTH})/8$ to avoid calculating the CRC for the reserved bits at the end of each flit.
- On each data cycle, `data_out`'s payload field is written, 1024 bits at a time, directly from `data_in` (minus reserved bits).
- On one extra “assembly” cycle after the 2 data cycles, `crc_state` is inserted into the assigned bits for CRC in `data_out` and `data_valid` is asserted.

This architecture only accepts one flit's worth of data every 3 cycles total (2 data + 1 assembly).

6.1.2. Waveforms

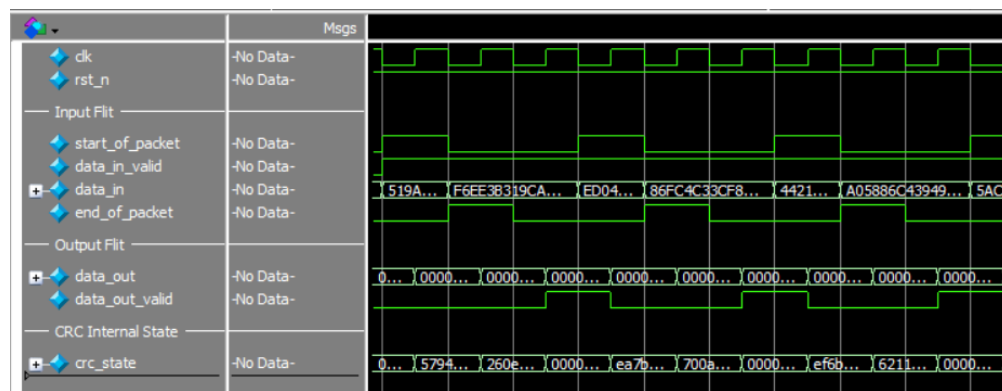


Figure 4: Three consecutive flits in 1024-bit configuration

6.2.2. Waveforms

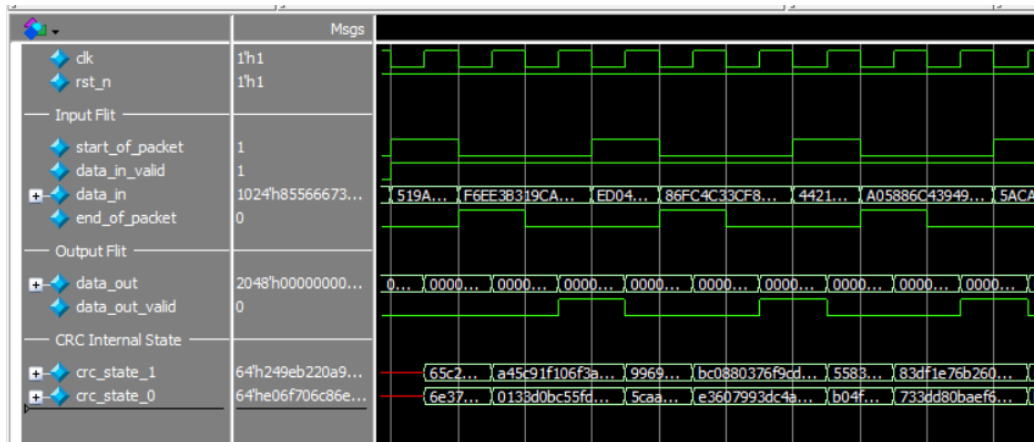


Figure 7: Three consecutive flits in 1024-bit configuration with split-processing

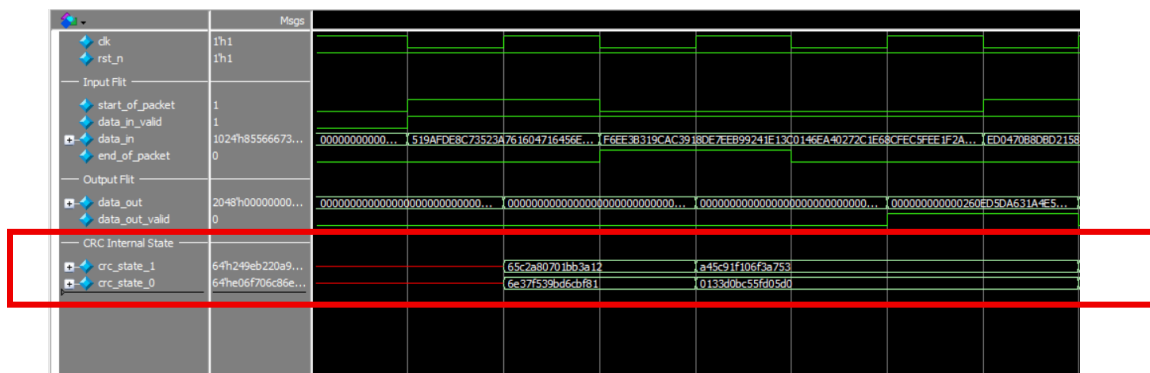


Figure 8: Internal CRC states for one flit in 1024-bit configuration with split-processing

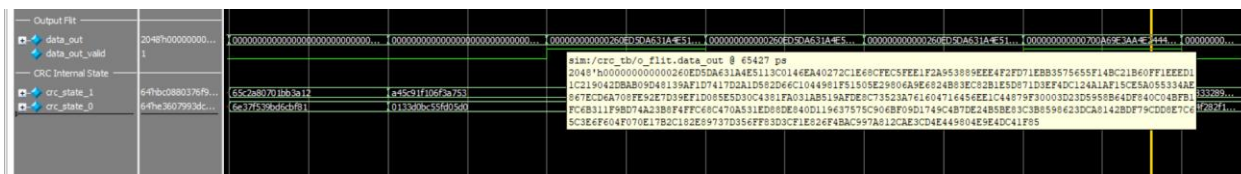


Figure 9: Output flit in 1024-bit configuration with split-processing

7. Golden Model (crc_golden_model.sv)

The golden model used for verification was the reference CRC function provided in Appendix K of the PCIe specification, which computes the CRC in a single flat, unpipelined calculation over the full 1936-bit payload in one cycle.

8. Summary of differences between 512-bit vs. 1024-bit (Split vs No Split) configurations

	512-bit config	1024-bit config (NO_SPLIT)	1024-bit config (SPLIT)
Cycles to receive one flit	4	2	2
crc_calc call width	512 bits, once/cycle	1024 bits, once/cycle	512 bits, twice per 1024-bit word (pipelined)
Combinational depth/cycle	1x 512-bit GF(2 ⁸) fold	1x 1024-bit GF(2 ⁸) fold	2x 512-bit GF(2 ⁸) fold
Assembly overhead	+1 cycle	+1 cycle	+1 cycle